

ENGINE — does the simulator do what Pokémon does

Owns: engine/medicham2-browser.js , engine/tag_dex.js , data/abra-tags.js , tests/test-mechanics.js , tests/walk_tags.js , tests/test-engine-diff.js , tests/test-game-diff.js , tests/interaction_matrix.js , tests/test-interaction-matrix.js , tests/mechanics_rank.js

Four instruments, and none substitutes for another:

file	asks	structurally cannot see
test-mechanics.js	is ONE mechanic live	tag x tag
test-engine-diff.js	is ONE HIT's damage right	every turn counter
test-game-diff.js	do the two engines hold the same STATE after every turn	damage magnitude
test-interaction-matrix.js	does every carrier x reactor pair resolve the way the official engine says	anything the generator refuses to emit — printed on every run

Its one number: mechanics live. **It must never go down.**

May not: claim a strength gain (that is SEARCH, gated by MEASURE), change what board.js *means* by a feature, or land during a fit or self-play run.

<!-- GENERATED: engine/status.js -->

```
ENGINE — does the simulator do what Pokémon does
181/186 probed mechanics live, 5 missing (census 2026-08-04 23:56)
missing:
  move      needsTargetToAttack    Avalanche doubles after being hit
  ability writesAccuracy            No Guard makes an 80%-accurate move land on a losing roll
  ability accuracyMod              Sand Veil makes the attacker miss a roll it would have hit
  ability untagged                 Marvel Scale raises Defense while statused
  move      reordersTurn            After You lets the partner move next
1/150 differential comparisons disagree with Showdown (2026-08-04 23:56)
chesnaught woodhammer -> mimikyu: showdown 0-0, medicham 120-130 (56 uses)
a differential hit is NOT in the census count above — the census probes what someone thought to probe
tag coverage: 148/180 probed, 32 unprobed
```

stamped 2026-08-05 00:26

<!-- /GENERATED -->

The working rule

A mechanic is not open work until a probe fails on it. Everything in the generated block above came out of an artifact; anything in the hand list below is a claim about the engine that nothing checks. The job of that list is to empty itself — each item becomes a probe in tests/test-mechanics.js , and from then on the census carries it and the line disappears from here.

That is the whole reason the census count may never fall: it is the only number in the project that a human cannot quietly soften.

THE FIVE MISSING MECHANICS, EACH WITH ITS REASON. 2026-08-05.

A declared gap with a reason is a finished item; an undeclared one is not. The census carries all five; this is why each is still there. There were 8; Air Lock came off it as WIRE 78, and conditionalPower and needsUntrackedState came off together as WIRE 83 — the "blocked on the derivation" verdict on both was retired by teaching tag_dex to read basePowerCallback and onBasePower arithmetic directly, exactly the pass those rows said it would take.

#	mechanic	by DECISION or by OMISSION	why
1	needsTargetToAttack — Avalanche doubles after being hit	DECISION, unchanged	The probe asks for a rule that does not exist. Avalanche doubles when the user was damaged BY THAT TARGET THIS TURN; dmgRange is handed no turn state and must not invent any. 13 corpus uses. The tag's other nine members include Sucker Punch (6,673), already fully modelled through failsIfTargetNotAttacking, so the tag is not inert — only this member is
2	writesAccuracy — No Guard	OMISSION, blocked on a SIGNATURE, costed rather than gestured at	moveAccuracy(id, field) takes neither body. 11 call sites across 4 files; two of them (board.js, position_features.js) are not ENGINE's and a signature change there is a feature-vector change, which is the refit edge MEASURE owns. The compatible shape is moveAccuracy(id, field, att, def) with both optional. It is a deliberate pass, not a one-liner
3	accuracyMod — Sand Veil	OMISSION, same signature, same pass as #2	—
4	untagged — Marvel Scale raises Defense while statused	OMISSION, blocked on the derivation	36 uses. It has NO tag at all: the mechanic is an onModifyDef conditioned on pokemon.status, and no derivation in tag_dex describes a conditional stat multiplier. Wiring it by name is what this file spends its time deleting. The Air Lock, Mummy and now WIRE 83 corrections all say the same thing about this row: "no artifact" is a claim about the DERIVATION and should be re-tested, not inherited
5	reordersTurn — After You lets the partner move next	OMISSION, and it is the nearest one to landing	The param IS usable — {sends: 'next'} / {sends: 'last'} — and After You (151) + Quash (156) are pure reordering. What blocks it is Instruct (162), which carries the identical {sends: 'next'} and does something completely different: it makes the TARGET use its last move again. Giving Instruct After You's behaviour would be a new wrong number, and nothing in the artifact tells the two apart. The derivation needs to distinguish them first

THE GENERATED INTERACTION MATRIX — tests/interaction_matrix.js + tests/test-interaction-matrix.js

Will: "Basically all the tags on moves and stuff should trigger all the flags on abilities and types and etc and have it flow from there", "we def need interactions thats the whole point and multi turn things like tailwind and trick room", "the interactions should be pretty formulaic now that we have all the tags and such." He is right that it is formulaic. The generator is the formula; the runner plays every case in medicham2 and in the official pinned engine and authors no expected outcome.

THE SIZE, AND THE COVERAGE, HONESTLY.

theoretical cross product, no filter at all	8,506 — flag 7,870, type 480, field 156
---	---

emitted at --full	1,640 — flag 1,171, type 313, field 156
by layer	secondary 848, legality 379, damage 323, immunity 83, targeting 7
LIVE (the reference engine's two arms differ, so the mechanic can fire)	1,008
INERT (the reference engine behaves identically with and without the reactor)	503
SATURATED (the control arm KO'd, so a damage ratio is clamped)	110
KO-TIMING (a damage-magnitude question — test-engine-diff.js owns it)	27
THREW	2
medicham2 agrees with the official engine on	940 of 1,008 — 93.3% (was 79.4% on the flag axis and 19.2% on the field axis when the instrument was first pointed at them)

EVERY DROP IS NAMED AND COUNTED AND PRINTED ON EVERY RUN. A silent cap reads as "covered everything", which is this project's signature failure. The largest buckets are carrier-does-not-aim-at-a-foe (400 — self- and side-targeting moves), carrier-is-a-die (662 across five accuracies plus chance secondaries — trap 2, a miss is luck and both engines must not be compared on it), no-control-carrier, reactor-not-in-format (Iron Barbs, Tangling Hair, Lingering Aroma and Perish Body have **zero** species in Champions), holder-immune-by-chart, and layer-unclassified.

FOUR THINGS A GENERATED CASE MUST KNOW, and the previous sampled version (82 pairs) got each wrong.

- 1. WHICH SIDE THE REACTOR STANDS ON.** linkage.contact.abilities holds Rough Skin AND Tough Claws. The sampled matrix staged every reactor on the DEFENDER, so Tough Claws, Long Reach, Unseen Fist, Iron Fist, Sharpness, Strong Jaw, Punk Rock and Poison Touch were all cases in which the mechanic could not fire — **and every one of them read as agreement.** The side is derived from the reactor's own tags and a reactor whose tags do not decide it is dropped as side-unknown.
- 2. WHICH RESOLUTION LAYER IT TESTS,** from docs/TAGS.md's own table. The layer decides the EVALUATOR, and that is not bookkeeping: the state comparator is blind to the DAMAGE layer by construction, so a halvesTypeDamage case handed to it comes back INERT forever and looks covered.
- 3. WHETHER THE PAIR CAN ACTUALLY MEET,** answered by the REFERENCE engine rather than by us. Every case is played **four** times — with the reactor and without it, in each engine. If Showdown's two arms are identical the case is INERT and is never counted as agreement. *Identical results across a varied knob mean the knob is unwired* — applied to the HARNESS as well as to the engine.
- 4. WHAT WAS DROPPED.** See above.

THE MULTI-TURN HALF IS GENERATED TOO, and the answer to "can the matrix drive it" is YES for the FIELD. A pair cannot reach a sequence, but the persistent field effects cross-product **with each other:** setsWeather, setsTerrain, reversesSpeed (Trick Room), doublesSideSpeed (Tailwind) and halvesDamage (the screens) are all derived from tags that mean *"this outlives the turn it was clicked on"*. Each of the 156 ordered pairs becomes an **eight-turn script** — A lands on turn 1, B on turn 3, everything idles to expiry — and every counter is compared at every turn. That is literally *"Trick Room was up and then a Tailwind landed"*, generated rather than typed. **156 of 156 now agree;** it was 30 of 156 when it was first run.

FOUR THINGS THE HARNESS GOT WRONG BEFORE THE ENGINE DID, and each is written into the code:

- **Protect has EIGHT PP.** A nine-turn field script ran every idle body out and made it STRUGGLE on turn 9 — three bodies suddenly damaged in 126 of 156 cases, an 80% "divergence" rate that was entirely the

harness. Eight turns still covers every expiry in the set.

- **Sturdy on the bulkiest Fighting-weak body.** `closecombat -> chopleberry` read a damage ratio of 56.7% against a true 50% because the control arm's overkill was stopped at 1 HP by **Bastiodon's slot-o ability**, so the "full damage" the ratio divided by was not the full damage. Every body that is not the one under test now gets an inert control ability and no item — `bare()` applied to a generator.
- **A body that faints is REPLACED at full HP**, so a lethal hit reads as a loss of ZERO. Strong Jaw's x1.5 came back as a ratio of 0.000, which looks exactly like a boost wired backwards.
- `stripIdentity` **deleted its own evidence.** Blanking `ability` on all four slots for the inertness test removed the ONLY witness Mummy and Wandering Spirit have. Only the body under test is stripped.

WHAT IT FOUND — ten wires, and not one was reachable from a single-mechanic probe.

#	found as	the bug	wire
1	24 of 156 field cases at once	Grassy Terrain never set a terrain. It carries <code>perTurnHP</code> for the terrain's own heal and that branch sits above the terrain branch in <code>playerAction</code> , so the one terrain move that also heals was the one the engine could not set — and the other three worked, which is why nothing noticed	72
2	the residue of #1	Grassy Terrain's 1/16 heal , derived from the terrain move's own tag	73
3	<code>sandstorm + grassyterrain</code> , the last pair standing	the sandstorm chipped on the turn it expired — five ticks where the official engine deals four. Visible ONLY as a pair, because the grassy heal is exactly the 1/16 the sand takes, so the two cancel and the extra tick is the only HP left on the table. The counter was never wrong , which is why nothing had caught it	74
4	<code>psychicnoise -> liquidvoice</code> , ratio medi 1.000 vs sd 0.375	<code>convertsMoveType.converts</code> names either a TYPE or a FLAG and only the type half was read, so Liquid Voice (346 uses) was completely inert	75
5	<code>psychicnoise -> soundproof</code>	<code>immuneToMoveClass</code> had one consumer per stage-3 mechanism instead of one per STAGE. A Soundproof body took zero damage and still got two turns of Heal Block. docs/TAGS.md already says <i>"an immune target takes nothing — not the damage, and not the secondary"</i>	76
6	<code>roar -> throatchop</code>	the Throat Chop silence was checked in the ATTACK branch and in chooseAction — one class of action out of a dozen. Roar is a sound move that resolves down the <code>phaze</code> branch, so a silenced body phased anyway	77
7	census row, not the matrix	Air Lock / Cloud Nine. See below — the "no artifact to wire from" verdict was wrong	78
8	<code>strengthsap -> suckerpunch</code> , medi's own arms identical	<code>statChangeInCode</code> with on: 'target' had a READER and no CLASSIFIER. WIRE 67 put the reader inside the pivot branch because Parting Shot was the case it was written for, so Strength Sap (637 uses) resolved to <code>kind: 'pass'</code> — a wasted turn	79
9	12 cases at once	Mummy and Wandering Spirit. Both grounds for filing them retired — see below	80
10	23 cases at once, at <code>--full</code>	the secondary that boosts the USER. The block read <code>status</code> , <code>targetBoosts</code> and the flinch and never <code>selfBoosts</code> , so Trailblaze, Aqua Step, Flame Charge, Rapid Spin, Torch Song, Aura Wheel and Psyshield Bash landed their damage and left the user's stages alone. 12 moves, 1,199 corpus uses	81

DEPTH IS A KNOB AND THE DEFAULT IS NOT THE WHOLE MATRIX. `--depth=N` takes the N most-clicked carriers per (key, reactor); `--full` takes all of them and the drop ledger counts what a depth cap excluded under `depth-cap`. Every number above is `--full`. **Depth matters:** three of the ten wires above are invisible below depth 6, and WIRE 81 needed `--full`.

What is left, and why each one is left

- **The variable-power family — Hard Press, Gyro Ball, Reversal, Steel Roller, Beat Up (35 of the 68 remaining cases).** These have a `basePowerCallback` and `MC.moves[id].bp === 0`, so `hasPower()` rejects them and they deal literally nothing. They are the same thing the census reports as

`needsUntrackedState` and `conditionalPower` MISSING, seen from the other instrument. The tag says `{needs: "speed ratio -- computable, not wired"}` and `{conditional: true}` — prose, not a rule — so wiring them means teaching `tag_dex` to read a `basePowerCallback`, which is a pass of its own.

- **Beak Blast (10 corpus uses).** It burns anyone who makes contact BEFORE the move fires — a charge-turn punish with no shape in this engine and nothing in the artifact that describes it.
- **The drain-heal / contact-punish ORDER (8 cases).** `medicham2` pays the Rough Skin toll and then applies the drain heal, so a full-HP attacker is healed back over the toll; Showdown drains during the move and tolls after. A real ordering defect, small, and named here rather than left.

Hand list — found by differential testing, not yet probed

EMPTY except for Rivalry, and Rivalry has never been probeable. Everything else that was on this list has become a probe and the census now carries it. That is the list doing its job.

- **Rivalry** — `x1.25` into the same gender, `x0.75` into the opposite, `x1.0` if either is genderless. Wholly absent. Blocked on data, not on will: `MC.mons` carries no gender and `buildMon` returns none, and `data/engine-data.js` belongs to MEASURE. Its `damageBoost` tag carries a bare `mult:1.25` with no condition and 43 other members including Blaze 1.5 and Slow Start 0.5, so it cannot be wired from the artifact as it stands. The differential can no longer see it either: CONTROL FIX 6 sets `gender: 'N'` on both sides, because `gender: ''` made Showdown roll one off the battle seed and MEDICHAM has none — a seed-dependent `x0.75` nothing could match.

THE THIRD INSTRUMENT — `tests/test-game-diff.js`, built 2026-08-04

Will: *"yeah we def need interactions thats the whole point and multi turn things like tailwind and trick room."* Neither existing instrument can reach that, and the reason is structural rather than an omission:

instrument	what it asks	what it structurally cannot see
<code>test-mechanics.js</code>	is ONE mechanic live	tag x tag. 176 tags individually verified says nothing about any pair
<code>test-engine-diff.js</code>	is this ONE HIT's damage right	every turn counter — Tailwind expiring, Trick Room toggling, a screen running out, an Encore ending
<code>test-game-diff.js</code>	do the two engines hold the same STATE after every turn	damage magnitude (that is the file above), and everything in its own <code>NOT_COMPARED</code>

It plays a fixed action script in `medicham2-browser.js` and in the official pinned Showdown engine, compares a dice-independent projection of the whole state after every turn, and reports the **first** turn they part. Artifact: `data/game-diff.json`.

TWO MODES, AND THE SPLIT IS THE POINT. `--pairs` GENERATES its cases from the tag artifact — the cross product of a linkage key's carrier moves against its reactor abilities — and never authors an expected outcome, because the official engine supplies it. That is the half that makes it safe: every one of the roughly twenty-three wrong probes this project has produced was a human writing down what should happen. A cross product can never reach a SEQUENCE, though, so the scripted multi-turn games are the other half. Neither substitutes for the other, and it is written in those words because the next person will otherwise try to make one do both.

IT DEPENDED ON PRIORITIES #44 AND THAT IS WHY #44 LANDED FIRST.

`reactorsTo('contact').moves` returned 152 moves that CARRY contact — Fake Out, Close Combat, Flare Blitz. Those are attackers. The moves that actually REACT — Spiky Shield, Baneful Bunker, King's Shield —

were not in the index at all. `tag_dex.js` now emits `carrierMoves` and `reactorMoves` separately, the reactor side derived by the SAME handler probe the abilities and items use, and `moves` is GONE rather than aliased: an alias keeps every existing misreading working silently.

WHAT IT DOES NOT COMPARE is printed on every run and is the honest half: HP amounts (the two engines roll their own dice), accuracy misses, chance secondaries, crit, a reactor whose effect is a roll, a KO the two engines time differently (a DAMAGE question, which belongs to `test-engine-diff`), the `protect` volatile (both have it and clear it at different points in the turn), Showdown-only volatiles, and PP. Stat VALUES are **aligned** rather than excluded, and that is a control that has to hold: unaligned, the two engines disagree about SPEED ORDER and about who survives a hit, and both read as rule divergences.

THE INJECTED-DIVERGENCE PROOF RUNS FIRST AND THE FILE REFUSES TO REPORT WITHOUT IT. One extra turn of Tailwind is planted on turn 2 of a clean game; the comparator must catch it, at turn 2, on `.field.tailwindA`. A silent zero is a broken comparator, not a clean engine.

What it found, in order

#	found by	divergence	verdict
1	scripted game 3	<code>weatherTurns medi=4 sd=3</code> on turn 2	REAL. A weather move clicked into its OWN weather REFRESHED the clock; Showdown fails it. WIRE 64 , weather and terrain together
2	pair matrix	<code>closecombat -> anything : boosts.def medi=-1 sd=0</code> , six pairs at once	REAL. A move blocked by Protect still paid its self-drop. WIRE 65
3	pair matrix	<code>partingshot -> soundproof : medi switched</code> , Showdown did not	REAL, two bugs. A blocked Parting Shot still pivoted (WIRE 65), and <code>immuneToMoveClass</code> lived only in <code>dmgRange</code> , so a Soundproof body took a sound STATUS move (WIRE 66)
4	pair matrix	<code>partingshot -> stancechange : boosts.atk medi=0 sd=-1</code> , three pairs	REAL. Parting Shot's <code>-1 Atk / -1 SpA</code> was the documented unmodelled half, 7,184 corpus uses. WIRE 67 , from a <code>statChangeInCode</code> derivation that now reads the literal boost object out of the handler
5	pair matrix	<code>fakeout -> toxicdebris : hazards.toxicspikes medi=null sd=1</code>	REAL. <code>punishesAttacker.hazard</code> had "nowhere to land" until WIRE 41 gave each side an <code>hz</code> bag. WIRE 68
6	pair matrix	<code>roar -> soundproof : medi phased</code> a Soundproof body	REAL. Roar IS a sound move and WIRE 66 had not reached the phaze branch
7	pair matrix	<code>encore -> stancechange : vol medi=["encore"] sd=[]</code>	REAL. Encore against a target that has never moved has nothing to repeat. WIRE 69
8	pair matrix	<code>wavecrash -> mummy / -> wanderingspirit</code>	REAL AND NOT FIXED. Contact REWRITES the attacker's ability. Both carry only <code>contactPunish</code> and neither has a param for it; o corpus sheets between them. Filed

FOUR OF THE TEN "DIVERGENCES" IT REPORTED FIRST WERE THE HARNESS, and each was fixed rather than excused: a bench index that meant different bodies in the two engines (trap 1 broken by the harness itself), an ally target emitted as a positive slot so `battle.choose` silently REJECTED the turn and froze the reference engine, a `benched` list that counted fainted bodies on one side only, and a reactor staged with Protect as its only move so the interaction under test could not happen at all. The last is Lesson 5 in a generator: **ask what the target would do if the mechanic did not exist.**

WEATHER — THE WHOLE SURFACE, AUDITED AT ONCE. 2026-08-04.

Will: "Weather is something that is the deciding factor in like every game so we need to get it bulletproof." It had been found broken four separate times in one day by four different routes, which means it was being found by luck. Batch 8 of `tests/test-mechanics.js` probes every path at once, and every probe in the batch declares its arms.

path	probe	result
setting, by move	<code>setsWeather</code>	LIVE, unchanged
setting, by ability on entry	<code>weatherSetter</code>	LIVE, unchanged
setting, by a MEGA's ability	<code>megaWeatherSetter</code> new	LIVE: base Charizard (Blaze) sets CLEAR, Charizard-Mega-Y (Drought) sets sun
duration and expiry	<code>weatherDuration</code> new	LIVE: sun on turn 1, still sun after 3 idle turns (Flamethrower 74), CLEAR after 4 (49)
the rocks	<code>extendsDuration</code> new arm	WAS MISSING — WIRE 70. The tag has carried <code>toTurns: 8</code> all along and only the SCREEN branch read it; the weather branch wrote a literal 5, so Heat, Damp, Smooth and Icy Rock were inert on the one mechanic they exist for
the rocks, on the other three routes	<code>test-weather-duration.js</code> new	WIRE 70 WAS ONE BRANCH OF FOUR — WIRE 71. See below
offensive multipliers, both directions	<code>weatherDamageMult</code> new	LIVE: Flamethrower clear/sun/rain 56/84/27, Surf 75/37/112
defensive multipliers	<code>weatherDefenceMult</code> new	LIVE: Shadow Ball into a Rock 31 to 21 in sand (into a Water 40 to 40), Earthquake into an Ice 157 to 106 in snow (into a Fire 246 to 246)
residual	<code>weatherChipImmune</code>	LIVE — sand 1/16, Rock/Ground/Steel and the tag's own immunities exempt, snow chips nothing
accuracy, both directions	<code>weatherAccuracy</code> new	LIVE: Thunder clear 70, rain 100, sun 50
Weather Ball	<code>weatherBall</code> new	LIVE: into a Gengar, clear 0 (Normal is nothing to a Ghost), rain 144, sun 144, sand 96, snow 96
Solar Beam	<code>chargeSkippedByWeather</code>	LIVE, unchanged
Aurora Veil	<code>failsWithoutWeather</code>	LIVE, unchanged
weather speed abilities	<code>speedCond</code> + <code>speedCondWrongWeather</code> new	LIVE with the WRONG-SKY arm: Swift Swim in rain 135 to 270, in sun 135
Solar Power	<code>solarPower</code> new	LIVE: in sun Flamethrower 84 to 126, Earthquake 37 to 37, no sun 56 to 56
Air Lock / Cloud Nine	<code>weatherSuppression</code>	LANDED — WIRE 78, and the "nothing to wire from" verdict was wrong. See below

TWO PROBES IN THE BATCH WERE WRONG BEFORE THE ENGINE WAS, WHICH MAKES TWENTY-THREE. The first Weather Ball probe fired it at a **Garchomp**, which is Dragon/**Ground** — so the sand form (Rock) is RESISTED, 100 BP at x0.5 is the same number as 50 BP at x1, and `sand 43 vs clear 44` read exactly like a dead knob. The engine was right and the type chart was doing its job. The first `megaWeatherSetter` control was a plain **Charizard**, and `buildMon` hands a Pokemon its USAGE item — which is a Charizardite Y, so the "base forme" arm was already a mega and already set sun. That is the original Choice Scarf mistake, verbatim, seven months later.

THE VOCABULARY, GREPPED FOR SURVIVORS. Three copies of the Showdown-name to engine-word map remained. `engine/medicham2-browser.js:182` is the canonical one behind the exported `weatherId()` . Two more sat inside `engine/tag_dex.js` 's `weatherScaled` and `weatherSetter` derivations, were identical to each other and **not** to the display map twenty lines above them (`hail` was `hail` in one and `snow` in the others). Both now read one `W2ENGINE` , and regenerating produced no change beyond the session's intended 35 entries — which confirms the two were already agreeing and the consolidation is a no-op today and a guarantee next month. `engine/board.js:1190` is the fourth and is **NOT ENGINE's**: it is a refit trigger (14 of 58 feature columns move) and MEASURE has the patch measured and deliberately reverted in `docs/MEASURE.md` section 11. Any further board.js weather defect is filed there, not fixed here.

WIRE 78 — AIR LOCK AND CLOUD NINE. "No artifact to wire from" was a claim about the DERIVATION.

The previous pass recorded this as MISSING-declared with "`cloudnine carries untagged and airlock has no artifact entry at all, so there is nothing to wire from.`" That was a true statement about `data/tags.json` and a false one about the dex, and the difference is the whole lesson:

Showdown does not express weather suppression through a HANDLER. It is a flat property on the ability object — `suppressWeather: true` . Every derivation in `tag_dex.js` probes a handler NAME (`onStart` , `onImmunity` , `onModifySpe` , ...), so a property-shaped fact was invisible to all of them, and the gap read as *not derivable* when it was *not looked for*. `dex.abilities.all().filter(a => a.suppressWeather)` returns **exactly two**, `airlock` and `cloudnine` , and Delta Stream carries the same property set to `false` and is correctly excluded by truthiness. The tags diff, excluding `uses` , is exactly those two entries.

IT IS NOT "CLEAR THE WEATHER". The sky is still raining and the clock still runs; what stops is every READ. So the suppression is a field flag recomputed at the top of every turn from whoever is standing there, and it gates: the damage multipliers (via one shadow at the top of `dmgRange` , so every weather read below it is covered without a gate per site), Weather Ball's type, the weather accuracies, Solar Power and Orichalcum Pulse, the snow/sand defence bumps, the weather-speed abilities, the sandstorm chip, the Solar Beam charge skip and Aurora Veil's legality. Clearing it instead would let a second Drizzle re-set it and would make the Veil's failure look like the absence of snow rather than its suppression.

THE EXPOSURE WAS MEASURED BEFORE THE WIRE, because it decides how much machinery this deserves. Air Lock is ZERO — its only carrier is Rayquaza and Rayquaza is not in this format. Cloud Nine has **two** carriers that are, **Altaria and Drampa**, and **18 declared sheets across 40,595 stored games**. Small, derived anyway, because the cost was one derivation block and the alternative was a census row asserting the engine cannot know.

A pure `dmgRange` call still cannot see an Air Lock ALLY — it is handed two bodies — and that is stated in the code rather than silently equivalent. The battle loop is the only caller that can, and it is the one that sets the flag.

WIRE 71 — WIRE 70 FIXED ONE BRANCH OF FOUR, AND THE PROBE COULD NOT SEE THE OTHER THREE

Weather is set **four** ways in this format, and each had its own branch:

#	route	site	read the rock?
1	an ability on switch-in — Drought 899 uses, Drizzle 3,075, Sand Stream 1,716, Snow Warning 1,561	<code>medicham2-browser.js:1524</code>	no — literal 5
2	a MOVE — Sunny Day 588, Rain Dance 919, Sandstorm 10, Snowscape 11	<code>:2149</code>	yes, since WIRE 70
3	MEGA evolution — Charizard-Y arriving with the stone	<code>rollout_leaf.js:186</code>	no — literal 5

#	route	site	read the rock?
4	a punish ability — the Sand Spit class	: 2629	no — literal 5

So a Torkoal holding a Heat Rock set **five** turns of sun by switching in and **eight** by clicking Sunny Day. Same held item, same sky, two answers, decided by how it arrived.

The probe that found WIRE 70 was staged on route 2 and passed on route 2. It was not a weak probe — it was a correct probe pointed at one of four bodies that can show the effect, which is the same shape as the mega bug that ran at 56% of sides against 85% and passed a non-zero check. A tag with four consumers needs a probe per consumer, or an assertion over the consumers as a set.

This one is the second: `tests/test-weather-duration.js` asserts the **invariant** — for a given sky and a given item, every route agrees — rather than four separate numbers. A fifth route added tomorrow that hardcodes 5 fails without anyone remembering to extend a list of four. The duration rule now lives once, in the exported `weatherTurns(weather, item)`, beside `weatherId`.

The two vocabularies meet inside it, and that is why it could not be a one-line read at each site:

`extendsDuration.extends` holds MOVE ids (`sunnyday`), because the rock's rulebook text names the move, while `weatherSetter.weather` holds ENGINE words (`sun`). WIRE 70's inline version compared the raw `extends` entry against `a.mv` — correct on the move branch by luck of spelling and silently never matching on the other three. Both sides now go through `weatherId`, so neither spelling is authoritative.

It is a small mechanic that decides whole games. 14 of 496 declared setters in the store carry the matching rock — Damp Rock on a Drizzle body is the common one at 6.2%, Heat Rock 2.2%, and neither Smooth nor Icy Rock appears at all. Three extra turns of rain is most of a game, and the population is small enough that no aggregate would ever have shown it.

A one-character bug caught in review, recorded because it is the shape that survives. The first cut of the punish branch read `m.item` — but `tg` is the **holder** of the punish ability and `m` is the attacker who set it off, so Sand Spit would have run for eight turns whenever the mon that hit it happened to carry a Smooth Rock. Right function, right tag, wrong subject.

The probe was run against the pre-fix engine and failed 4 of 60, then passed 60 of 60. A test written after a fix that is never shown failing is an assertion about the code as it stands.

And the unit half would not have caught the original bug. `weatherTurns` returns 8 correctly whether or not the switch-in branch calls it. `test-weather-duration.js` therefore builds a real Torkoal, runs the real `applyEntryEffects`, and reads the real field — with a counter asserting the probe ran 8 times, which is what caught the build failing silently: `buildMon`'s override bag is keyed by **species** (`{torkoal: 'Heat Rock'}`), so the natural-looking `{item: 'Heat Rock'}` was ignored and the mon kept its dataset default of Charcoal. Without the counter that is a probe measuring the wrong item and reporting a pass.

PRIMORDIAL SEA AND DESOLATE LAND ARE UNIMPLEMENTED BY DECISION, NOT BY OVERSIGHT. Will, 2026-08-04: *"The primordial and desolate stuff aint in this regulation so just make a note to deal with that if kyogre gets added."* **o occurrences in 339,483 boards.** `tag_dex`'s `W2ENGINE` maps both onto plain rain and sun, which is wrong for the real mechanic — they cannot be replaced and they nullify the opposing type entirely — and correct for a format that cannot produce them. **The trigger condition is a primal Kyogre or Groudon entering the format.** Recorded here so a regulation change surfaces it rather than someone rediscovering it as a bug.

THE ARMS PROTOCOL — the hollow detector, finished. 2026-08-04.

The previous pass built the structural detector (a probe that READS THE SOURCE) and **costed** the other half rather than doing it: a probe with ONE arm, whose reading an engine with the mechanic DELETED would also produce. That is what made the Disable probe a false LIVE for as long as it existed, and the heuristic beneath it — count LIVE probes whose `detail` carries two equal numbers — is a heuristic precisely because `detail` is prose and cannot tell an ARM from an annotation.

It is done. A probe may now return `arms: {control, test}`; the harness asserts `control !== test` structurally, with no parsing and no judgement, and a probe whose arms agree is marked **HOLLOW** and fails the file exactly as a source grep does.

THE OPT-IN IS NOT A HOLE, BECAUSE THE OPT-OUT IS RATCHETED. `unarmed` is written to `data/mechanics-census.json` and **may fall and may never rise**. A new probe written without arms fails the file. The existing ones convert at whatever rate a pass can afford — which is the cheapest version that closes the hole rather than costing a day up front, and it is exactly what the previous pass's costing asked the next one to decide with the number in front of it.

Seven were converted in this pass and every one was a genuine one-armed probe, exactly as the heuristic said: `lowersUser ( def -1 spd -1` is also what an engine that dropped the user on EVERY attack prints — the control is now Brave Bird, which must leave the stages alone), `recharge`, `statChangeInCode`, `boostsTarget`, `clearsBoosts`, `cantUseTwice`, `statusCategory`. Ten more were written armed. **The flat-arms heuristic fell 7 to 2, and both survivors are annotations rather than arms.**

Filed by the game differential, not fixed

- **~~Mummy and Wandering Spirit rewrite the ATTACKER's ability on contact.~~ LANDED as WIRE 80, and BOTH grounds for filing it were wrong.** The filing said (1) *"neither has a param for the rewrite, so there is nothing to wire from"* and (2) *"o corpus sheets between them"*. (1) was true of the TAG and false of the DEX: both handlers state the whole rule in one call — `source.setAbility("mummy", target)` and `this.skillSwap(source, target)`, both gated on `checkMoveMakesContact` — so `tag_dex` now derives `rewritesAbilityOnContact` with a `mode` of `infect` or `swap`, matching **exactly three** abilities (the third is Lingering Aroma, 0 uses). (2) does not hold on the current store: the artifact's own counts read **mummy 41, wanderingspirit 58**. It matters more than 99 sheets suggests, because the ability is an INPUT to every later number — a Blastoise that walks into a Cofagrigus keeps being priced as a Torrent body for the rest of the rollout. That is the Knock Off lesson in CLAUDE.md, one field over. **The general lesson: "the artifact cannot express it" is a claim about the DERIVATION, not about the dex, and it has now been wrong twice in one session — here and for Air Lock.**
- **A KO the two engines time differently is a DAMAGE question**, and `tests/test-engine-diff.js` owns it. One pair (`bitterblade -> sharpness`) is excluded on that basis and COUNTED, never dropped.
- **The `moveAccuracy` table in `medicham2-browser.js` is a hand-typed 35-move literal** and carries neither Triple Axel nor Population Bomb, so it returns 100 for both. WIRE 59 reads the per-hit accuracy out of the `multiAccuracy` tag instead and says why in place, but the TABLE is the same class of hand list this file has spent the session deleting and it should be derived.
- **Strength Sap's HEAL is still absent (WIRE 79 landed only the Attack drop)**, and the receipt for it moved: the move used to be counted in `fails.healProcedural` because it resolved to `kind: 'heal'` with `heal: true`, and it now resolves to `affect`, so that counter no longer sees it. The heal scales off the TARGET's Attack and no artifact this engine reads carries it. Recorded here because a counter that quietly went down is exactly the shape this file exists to stop.

- **Grounded-ness is still not tracked, and WIRE 73 inherits the gap.** Grassy Terrain heals a Flying TYPE correctly (skipped) and heals a **Levitate** body it should not, counted in `fails.terrainHealUngrounded` . Same declared gap as `fails.hazardUnresolved` , one field over.

The three ratchets — 2026-08-04, PRIORITIES #40 / #40a

All three were red **before** this session and two of them **crashed** rather than failed unless `SHOWDOWN_PATH` was set, which is most of why nobody ran them. Both now exit **2** with one line: *NOT RUN — set SHOWDOWN_PATH*. A check that crashes is a check that gets skipped.

Ratchet	Was	Now
<code>test-mc-key</code>	RED, <code>medicham2-browser.js</code> : 5 -> 7	GREEN , back to 5. The two lookups were removed , not baselined
<code>test-effective-identity</code>	RED, 234 → 302 raw reads	GREEN , with 7 files declared and <code>medicham2</code> 's declaration pinned by an assertion
<code>test-no-silent-failure</code>	RED, 53 new silent catches	still RED at 40 , and none of them are in a file ENGINE owns

`--update` **was refused on all three, and that is the point.** Raising a ratchet is the opposite of what a ratchet is for. `test-no-silent-failure --update` would have laundered 40 catches belonging to MEASURE, SEARCH and OPS along with the 11 that were mine.

The two mc-key lookups both went through `pasteKey()` , this file's own resolver — the one the test's header already names as working. `bringIn` normalised `switchInForme.becomes` by hand and then indexed `MC.mons` ; `oneMegaPerSide` wrote out the `-mega` suffix strip that `pasteKey` already does. Neither was a behaviour change and both are strictly more capable (`pasteKey` has the flat rescan).

`test-effective-identity` **needed a third option, because it had only two and both were wrong.** Leave it red forever, or `--update` and launder `rollout_leaf.js` 's real violation with the rest. So a file whose raw reads are correct BY CONSTRUCTION now DECLARES that, with the reason, in the shape this project already uses for `RAW-STORE-OK` . Three rules keep it from being a mute button: the reason must be about construction, a declaration that can be pinned is pinned, and every declared file prints its count on every run. **The ratchet was re-verified to still bite** by dropping a throwaway file with two raw reads into `engine/` — it failed on it.

All 67 of medicham2's raw reads were walked, not asserted. 66 are live battle bodies this engine constructed; the one exception (`norm2(set.ability)` in `buildMonFromSet`) reads a parsed SHEET, which is the case the test itself names as correct.

What the walk found is the part that mattered. The construction claim was FALSE — see below.

What verifying the claim turned up: two mega-ability bugs

Both are probed in `tests/test-mechanics.js` . Census **100** → **102 live**, nothing fell.

1. `megaRowAbilityCase` . 85 of the 318 `MC.mons` rows key a mega and store `ab` in DISPLAY case — "Technician", "Huge Power", "Tough Claws" . `buildMon` copied that through, and every ability test in this engine is a lowercase literal (`att.ability==='technician'`). A body built from its MEGA ROW carried exactly the right ability and **not one line of it fired**: Mega Scizor's Bullet Punch read **52 where Technician makes it 78**. It hid because the OTHER door — base row + stone — goes through `megaAbility()` , which returns from a lowercase hand-written map, and because `board.js` overwrites the ability with `effAbility()` before its own damage call. Only the mega-keyed door was wrong: `position_features.js` , `sets.js` , `winProb2` called with a mega name.

2. `megaSheetAbility . buildMonFromSet read declaredAb || megaAbility(...)`, so a paste of *Scizor @ Scizorite / Ability: Swarm* built a `scizor-mega` body running **Swarm**. A team sheet lists the PRE-mega ability; this is the mega ability gap living in the engine rather than in `board.js`, which had already fixed its own half at `board.js:964`. **Two engines disagreeing about a FACT**, which CLAUDE.md names as this project's most expensive failure. Only the branch that swaps the key to a mega row overrides the sheet — a non-mega set still lets a declared Rough Skin beat the dataset's Sand Veil.

`tests/test-paste.js` **was asserting the bug and its assertion was inverted**. It read `geng.ability === 'cursedbody'` for a Gengar holding a Gengarite. Mega Gengar has **Shadow Tag** — the worked example in `test-effective-identity`'s own header. Confirmed three ways before the line was touched: Showdown's dex gives Gengar-Mega exactly one ability, `board.js effAbility` returns `shadowtag` for the same sheet, and `medicham2` now agrees.

A probe was wrong before the engine was, for the tenth time. The structural pin first reported three megas with the wrong ability — Garchomp, Lucario, Absol. Champions ships a **second mega per stone**: `garchompitez -> garchompmegaz`, Rough Skin instead of Sand Force. The loop kept whichever stone it saw LAST, which was the Z one. They are now excluded **with the reason**: `MC.mons` has zero `-mega-z` rows and the store has zero `itez` occurrences, so the engine cannot represent them at all.

The differential harness was silently dropping 12 rows, all of them the same Pokemon

Five catch blocks in `tests/test-engine-diff.js` returned a plausible `null` and said nothing, so a row that failed to BUILD and a row that was never sampled were the same event. That matters here more than almost anywhere: the headline is a RESIDUAL, and a silent drop shrinks the denominator without shrinking the claim. Now counted, named and written to `data/engine-diff.json` as `dropped_by_exception`.

Naming them earned its keep on the first run. At seed 20260804 `--n 400` it fires **12 times and all twelve are Bellibolt**. The throw is inside Showdown: Electromorphosis's `onDamagingHit` adds the `charge` volatile, and Charge's `onStart` reads a `source` that only the full move pipeline sets — `moveHit`, the entry point this harness must use, leaves it null. **Not a MEDICHAM bug**, and on the same layer boundary as the Disguise SUSPECT row, so it is recorded rather than papered over. It was invisible until this session.

`tests/walk_tags.js` had the same shape and worse: its three silent catches returned `null`, and a `null` prints as **NOT COVERED**, which the report calls "honest ignorance, not a pass". It was not honest — an engine that THREW and a tag with no handler produced the same word.

PRIORITIES #0 — the leaf's weather string. LANDED 2026-08-04, with a before/after.

The engine could SET weather and could not READ the weather it was handed. `board.weather` holds Showdown's `[-weather]` line, which is a MOVE name — and across **41,122 weather events in the whole store there are exactly four values**: `SunnyDay` 17,375, `RainDance` 11,355, `Snowscape` 6,304, `Sandstorm` 6,088. Every formula in `medicham2-browser.js` compares against `sun / rain / sand / snow`. `rollout_leaf.applyField` assigned the string straight through, so a mid-battle board's weather was truthy enough to suppress the mega-weather guard and meaningless to every multiplier.

What was exported, and why there. `SD2WEATHER` already existed in `medicham2-browser.js`, beside `SD2ENG` and `CODE_OF_STATUS`, under a comment saying naming conventions live there. It is now wrapped in `weatherId()` and exported from that file — **not moved**, and **no second map written**. The engine owns its own vocabulary; a copy in `rollout_leaf.js` is how `choiceLock` came to have two engines disagreeing. It is

idempotent, so the two paths that already spoke the engine's words (`weatherSetter` on switch-in, a weather move played inside a payout) are untouched, and an **unrecognised value returns '' and is counted in** `MEDI.fails.weatherUnknown` rather than passed through as a truthy string nothing reads. That counter is **0** over 10,000 payouts. `:1481` , `:1825` and `:2156` were routed through it too — `:1825` read `SD2WEATHER[_pun.setsWeather]` with no normalisation and would have silently failed on any capitalised value.

Parity, 250 corpus boards, both arms in ONE process at n=40, seeds fixed per board.

	before	after		
boards walked	250	250		
boards carrying a weather	130 (52.0%)	130		
<code>rolloutWinProb</code> different	—	77 (30.8% of all boards, 59.2% of the weather boards)		
boards moved with NO weather	—	0 ← the control		
mean \	Δ \	on the movers	—	9.98 pt , max 37.5 pt

By weather: sun 28/41 at 12.10 pt, rain 27/42 at 9.68, snow 7/15 at 9.11, sand 15/32 at 7.00. The snow and sand numbers are the right order for weathers whose main modelled effect is a defence multiplier. **The 41% of weather boards that did not move are n=40 quantisation and saturation, not inertness** — the direct counter below settles that a different question was asked of the engine.

The direct counter, because a parity delta is indirect. `battleTurn` wrapped, field read on turn 1 of every payout, 250 boards × 40:

```
before : 0 of 10,000 payouts began in a weather MEDICHAM can read (0.00%)
after  : 5,760 of 10,000 (57.60%) sun 2120, rain 1600, sand 1
```

Probed permanently by `boardWeatherLanguage` in `tests/test-mechanics.js` , which calls the real `applyField` and demands the resulting damage EQUAL the damage under the engine's own word, having first shown that `sun` beats clear on the same Flamethrower — `clear 56, 'sun' 84, as landed 84` . `applyField` is exported as a named test seam for it.

The two dead wires in `tests/test-tag-wire.js` — BOTH CLOSED, and only one was the engine

It is **GREEN**, 104 checks, 0 dead wires. Census **102** → **105 live** / **144** → **147 probed**; nothing fell.

- `typeImmunity` **was the PROBE, not the engine.** The wire staged Volt Absorb on a **Garchomp** — Dragon/**Ground**, which takes zero from Electric with no ability at all. So *"an Electric hit into Volt Absorb prices at zero"* was true of a Garchomp with `ability: 'none'` , and the heal could not fire because nothing was ever absorbed. It also left the absorber FREE with `moves: [ 'protect' ]` , so the engine chose Protect and blocked the hit — two independent reasons the arm could not pass whatever the engine did. Same shape as the redirection false alarm (a Dragon move at a Fairy type). Re-staged on **Milotic** with an explicit control arm, the unmodified engine reads `hp change: ability none -34, Volt Absorb +35` (a quarter is 35) — **exact**. New census probe `typeImmunityHeals` , LIVE on the engine as it already stood.
- `sealsMoves` **was the engine, and the consumer was unreachable.** The wire sat inside `playerAction` 's `kind==='status'` branch. Encore, Disable and Taunt carry a **volatile and no major status**, so `playerAction` classifies them as `affect` and control never arrived. Moved to where they

actually resolve (**WIRE 26**), and **the duration now comes from the artifact** — it read `encore? 3:taunt?3:1`, so Disable, which the tag says lasts **5**, got one turn. Encore now rides the **same** `_lock` **the Choice items use**, so a caller-SUPPLIED action is bound as well as a chosen one (the WIRE 24 rule, which nothing about Encore honoured); a Choice lock is never shortened by it.

And the Disable probe in the census had been a FALSE LIVE for as long as it existed. It ran ONE arm: the foe committed Rock Slide, was Disabled, chose freely, picked Earthquake, and the probe called the mechanic live. Remove the Disable click and it picks Earthquake anyway. **Identical results across a varied knob mean the knob is unwired.** Both arms print now, and the staging was inverted so the control REPEATS — Rock Slide is precisely the move the control does not repeat, which is why the probe read green while dead.

Filtering `me.moves` **was not enough, and the probe caught that too.** The priors sampler (`MC.priors[me.name]`) picks a move **by name** and never consults `me.moves`, so the seal leaked through the single most-used exit in `chooseAction`. Both arms clicked Dragon Claw until it was guarded. Disable also ticks at **end of turn** rather than in `chooseAction`, or a duration only counts down on turns the engine happens to be choosing — which in a rollout driven from outside is never.

My own Encore probe was wrong first, which makes eleven. It handed the foe a FORCED action on the pinned turn, and a forced action bypasses `chooseAction` — it measured the caller's obedience. Both probes now read `S.lastActs`, the engine's own record of what was clicked, because `_lastMove` is not written by every action kind and a pass leaves yesterday's move sitting there.

The HEALING CLASS — validated as a class, 2026-08-04. Census 105 → 107.

Will: *"lets validate all the healing tags like drain and hospitality and life dew and recover and regenerator."* It is a class and only one member (`drain`) had been fixed. All eight members now have a **behavioural, two-armed** probe and all eight are LIVE.

THE DIFFERENTIAL CANNOT SEE ANY OF THIS, and `tests/test-mechanics.js` **now says so in its own comment.** `test-engine-diff.js` compares one `moveHit` against one `dmgRange` — a single-hit DAMAGE number. Healing is HP over turns. A residual of 1/400 says nothing whatever about this class. Same statement `multiHit` carries, same structural reason.

The tag over-matched, and printing the membership first is what caught it

`tag_dex.js` derived `healsOnSwitchOut` as `a.onSwitchOut ? {heal: 1/3}` — **any** ability that does anything on the way out. Membership printed **before** wiring:

```
OLD derivation matched 3 : naturalcure, regenerator, zerotohero
NEW derivation matched 1 : regenerator {"heal":0.3333}
```

Natural Cure cures status and heals nothing; Zero to Hero forme-changes Palafin. Wiring the tag as it stood would have handed two abilities a 33% heal they do not have, on **227 corpus uses**. The derivation now READS the number out of the handler — `pokemon.heal(pokemon.baseMaxhp / 3)` — instead of assuming it. LESSONS §4 for the fourth time.

The regeneration was verified, not assumed, by the procedure this file already documents: after excluding `uses`, **exactly 2** entries in `data/tags.json` differ and both are the intended ones (411 entries' `uses` moved because the store grew; no feature reads `.uses`). `engine/feature_fixture.js --check` exits **0** afterwards, so **no refit is owed**.

What was WIRED versus what was already HARDCODED

Tag	Uses	Before	Now
healsOnSwitchOut	845	nothing at all — a Regenerator pivot was priced as a plain switch	WIRE 27, from the corrected tag, in <code>switchOut</code>
blocksHealing	196	nothing at all	WIRE 30, on a connecting hit, per target
passiveHeal	6,483	<code>if (m.item === 'leftovers') ... /16</code> — a name check	WIRE 29, from <code>passiveHeal.heal</code>
healsSelf / healsAlly	2,592	the dex blob's <code>fx.heal</code> , and <code>fx.target === 'allies'</code> for the spread — a second copy of the fact	the tag's own params; Life Dew spreads because it carries BOTH tags
drain	8,553	already tag-driven (WIRE 19)	+ the Heal Block gate
healsAtThreshold , healsAllyOnSwitchIn , perTurnHP	—	already tag-driven	+ the Heal Block gate where it applies

Routing `healsSelf / healsAlly` **through the tag is a behavioural NO-OP and that was checked move by move before the switch:** across all 14 members the tag's fraction equals the dex's exactly wherever both exist (`lifedew [1,4]` , `roost/recover/slackoff/softboiled [1,2]`), and where the tag says `heal: true` the dex carries nothing either. So nothing moved and **membership stopped being typed** — docs/TAGS.md invariant 3. A move added next regulation with `healsSelf` now works without an edit.

`heal: true` **is a boolean in a fraction's clothing and is left visibly unwired**, counted in `MEDI.fails.healProcedural` : Rest (full, plus the sleep), Synthesis / Moonlight / Morning Sun (weather-dependent), Wish (delayed), Healing Wish (the user faints), Swallow (needs Stockpile), Strength Sap (scales off the TARGET's Attack), Heal Pulse (91 uses, heals its target). They are deliberately NOT classified as `kind: 'heal'` — Rest's real click is the sleep and Strength Sap's is the Attack drop, and capturing them here would turn a partly-modelled move into a fully no-op turn.

`blocksHealing` **is the counter and it landed in the SAME pass, on purpose**

Wiring the healers without it makes every healer in the format strictly better than it is — a one-directional error. It gates **healing moves, the heal half of a drain (the damage still lands), a passive item tick, a pinch berry (which is not consumed either), and Leech Seed's return to the seeder**. It does **not** gate `healsOnSwitchOut` — that fires as the body leaves, and leaving ends the volatile — nor Hospitality, which heals on ENTRY. Both exclusions are stated in the code.

THREE CENSUS ENTRIES WERE HOLLOW — LIVE by SOURCE GREP, not by behaviour

The contradiction Will's brief pointed at turned out to generalise. `healsAllyOnSwitchIn` read `/hospitality|healsAllyOnSwitchIn/.test(src)` — it would have returned **LIVE for a mechanic that was commented out, renamed, or wired to the wrong body**. It is now behavioural and two-armed and reads `ability none 0`, Hospitality 43 (a quarter is 43). **A hollow entry is worse than a missing one, because it occupies a slot in a number that may never fall.**

Two more of the same shape were still LIVE by grep and were named here rather than quietly left: `priorityMod` and `weatherChipImmune`. **Both were converted on 2026-08-04 — see "THE HOLLOW PROBES" below. One was honest and one was hiding a dead wire.** `blocksBerries` and `disablesAttacker` were the same shape reporting MISSING; they are behavioural now too, and still MISSING.

The census-versus-tag-wire contradiction, and which one lied

tests/test-tag-wire.js **lied**. data/mechanics-census.json was right: Volt Absorb heals exactly a quarter and always did. The wire staged the absorber as a **Garchomp** — Dragon/**Ground**, already immune to Electric with no ability at all — and then left it FREE with `moves:['protect']`, so the engine chose Protect and blocked the hit. Two independent reasons the arm could not have passed whatever the engine did. See the dead-wires section above; the census probe `typeImmunityHeals` was written this session, watched fail on nothing, and reads `ability none -34, Volt Absorb +35`.

Where the fact lives, since it lives in an odd place. `voltabsorb`, `waterabsorb`, `stormdrain`, `poisonheal` and `healer` carry **no heal tag**; the heal is a param of `typeImmunity` and that is the one place it lives. Volt Absorb and Water Absorb are the same shape and both work.

Not done, and named so nobody assumes it was

- `leechseed` **and** `pollenpuff`. Leech Seed is **already tagged** — `perTurnHP {effect:'drain', per:8, on:'target', to:'user', immuneType:'Grass'}` — and already wired (WIRE 8), so the brief's "carries no heal tag at all" is not quite right; what it lacks is membership in the `drain` tag, and adding a second tag for the same fact is what invariant 2 forbids. **Pollen Puff (99 uses) is a real gap**: it heals an ally instead of damaging and carries nothing for it.
- `naturalcure` **is now** `untagged`. It genuinely cures status on switch-out and no derivation describes that. It lost a tag it should never have had; it still needs one it does not have.

Found red, NOT mine, NOT fixed — reported rather than filed

- tests/test-web-status.js **is RED BECAUSE MY CENSUS MOVED, and I did not run the fix**. It says `engine.live = 105` but `data/mechanics-census.json -> live = 107`. The fix is one command and the test names it: `node web/build-status.js`. It is a generator — it authors no number, it copies artifacts onto the board — but it writes into `web/`, which this pass was told not to touch. So it is reported here in full rather than left to be discovered: **WEB owes one command**.
- tests/test-no-silent-failure.js **is RED at 47 new silent catches (was 40), none in an ENGINE file**. The seven that appeared since the 40 were counted: `backtest_winrate.js`, `mag_bot.js` ×3, `miltank.js` ×3 and neighbours — MEASURE, SEARCH and OPS. The one match inside `rollout_leaf.js` (`movePriorFor`, line 216) is **pre-existing at HEAD** and was verified against `git show HEAD:engine/rollout_leaf.js`; it moved down seven lines because of a comment.
- `test-site-data-fresh` **and** `test-stadium-roster` **are RED and are MEASURE/WEB**. The first is the stale-fit list (`pory`, `xatu`, `nmf`, `slowking`); the second is the GURU hole, PRIORITIES #41.

A SECOND weather boundary exists, is the same bug, and is measured at ZERO exposure — FILED

`engine/position_features.js:291` builds `field.weather = B.norm(board.weather)` — the same untranslated move name — and hands it to `M.effSpeed`. The loss is exact and was measured, not argued:

```
swiftswim  clear 161  engine-word 'rain' 322  board-word 'raindance' 161
chlorophyll clear 161  engine-word 'sun'  322  board-word 'sunnyday'  161
sandrush / slushrush  identical
```

It is not fixed here, and the reason is a number. Over **400 corpus boards, 192 of which carry a weather**, **ZERO** had a weather-speed ability among the actives standing in its own weather (95% upper bound ~0.75%). The control holds: `MC.mons` carries **9** such rows — Excadrill, Swampert-Mega, Basculegion-F,

Venusaur, Vileplume, Victreebel, Overqwil, Beartic, Houndstone — so the mechanism is real and the event is rare, rather than the probe being broken. It is a latent hazard, and fixing it changes a **feature vector**, which is the refit edge MEASURE owns. `board.js:1190` holds a THIRD copy of the same map (`WEATHER_KIND` , sun/rain only), and `board.js` is not ENGINE's to edit.

Terrain was the same mismatch and it is now MEASURED AND FIXED — see the next section.

PRIORITIES #o's TWIN — THE TERRAIN VOCABULARY. LANDED 2026-08-04, with the exposure measured first.

The split ran through the middle of this engine, not only at its edge. Three writers, two vocabularies, nobody translating — and medicham2's own readers were on *both* sides of it:

Reader	wanted	got from the artifact	got from a Board
Hadron Engine (:631)	electric	electric ✓	electricterrain X
Grassy Glide (:97)	grassy	—	grassyterrain X
Psychic Terrain priority block (:144)	psychicterrain	psychic X	psychicterrain ✓

Measured on the shipped engine before anything was touched:

Surf under Hadron Engine	clear 99	'electric' 130	'electricterrain' 99
movePriority(grassyglide)	'grassy' 1		'grassyterrain' 0
priorityRefusedAbove	'psychic' Infinity		'psychicterrain' 0

So **Psychic Surge has never blocked a priority move** (its `terrainSetter` says `psychic`), and a board's `electricterrain` has never boosted or hastened anything. Both halves looked live and both were dead, in opposite directions.

Exposure, measured before the fix

- **The store holds exactly four values** over **1,845** field-start terrain events across 52,441 games: `Electric Terrain` , `Psychic Terrain` , `Grassy Terrain` , `Misty Terrain` . The translation therefore covers **100%** of what exists, the same way the weather one did.
- **1.98%** of the 8,759-game fit corpus carries a terrain at all (173 games, 199 events).
- **863 of 69,623 corpus boards — 1.24% —** carry a terrain by the Board's own key, against **48.1%** for weather. By value: electric 597, psychic 243, grassy 18, misty 5.
- **o of those 863** are found by the extractor that actually feeds the leaf. `miltank.js:781` and `rollout_r1.js:175` `do ['electric','grassy','misty','psychic'].find(t => board.hasField(t))` — a **THIRD** vocabulary, short words against a Board that stores long ones. **Leaf-side terrain exposure is exactly zero today and stays zero until those files' owners change them.** Filed below.

Parity, and the honest answer is ZERO with a reason

150 terrain boards + 150 no-terrain controls, `n=40` , seeds fixed per board, the board's own key handed in, pre-fix and post-fix engines run over the identical sample:

	terrain boards	control (no terrain)
paired	142	134
<code>rolloutWinProb</code> different	o	o

Zero everywhere, and this time it is not "the fix did not take" — it is measured inertness with three stacked causes, each of which is a number rather than an argument:

1. the 37 psychic boards were **already** reaching the one reader written for the board's spelling, so before == after there by construction;
2. the 101 electric boards can only be read by **Hadron Engine**, which has **0 corpus uses** and exactly one `MC.mons row ( raichu-mega-x );`
3. grassy is **18 of 69,623** boards and its only reader is **Grassy Glide, 3 corpus uses**.

The direction that *did* change — the artifact's `psychic` now blocking priority — has **2** corpus uses of `psychicsurge`. So the terrain **field** is a tiny lever in this format, and the vocabulary fix is worth having because it is correct, not because it moves the number.

The terrain MOVES are the lever, and they are where the pass paid. `psychicterrain` is clicked 114 times, `expandingforce` 182, `risingvoltage` 114 — an order of magnitude more than the abilities. Both were wired in the same pass (`setsTerrain`, `terrainScaled`, below) and both go through `terrainId`, which is what makes the translation load-bearing rather than decorative.

Whole-session parity, same 276 boards, session-start engine against now — this arm has no control by construction, because the sandstorm chip and Magic Bounce also landed, so it is decomposed by weather instead:

sandstorm	9/22 moved (40.9%)	<- the sandstorm residual
none	9/118 (7.6%)	<- Magic Bounce, the terrain moves, Expanding Force
raindance	3/50 (6.0%)	
sunnyday	0/59 (0.0%)	<- the control: nothing this pass touches sun
snowscape	0/27 (0.0%)	<- and snow is correctly NOT a chip in this generation

Mean $|\Delta|$ 4.20 pt on the terrain boards and 7.00 pt on the rest, max 8.8 pt.

`terrainId()` **is** `weatherId()` 's sibling and no second map was written. Same shape, same idempotence, same loud unknown (`fails.terrainUnknown`, which names the first value it drops). It is exported. Idempotence matters more here than it did for weather because **both** vocabularies genuinely arrive — the artifact's on a switch-in and the Board's at the leaf boundary. Probed permanently by `boardTerrainLanguage`, which asserts **both sites in both vocabularies**: `clear 99, 'electric' 130, as landed 130 and priorityRefusedAbove: clear Infinity, 'psychic' 0, 'psychicterrain' 0`.

THE HOLLOW PROBES — one was honest, one was hiding a dead wire

`weatherChipImmune` **was LIVE by grep and THE ENGINE HAD NO WEATHER CHIP AT ALL** The probe read `/icebody|weatherChipImmune|magmaarmor/.test(src)`. It passed on the word `magmaarmor`, which appears in this engine **once** — inside the **freeze**-immunity table at `:1097`, with nothing whatever to do with weather. So the census carried an immunity as working while the damage it is immune to did not exist: burn, poison, Toxic and Leech Seed all ticked at end of turn and **sandstorm did not**. Sand Stream is 1,705 sheets and the store holds 6,167 sandstorm events. CLAUDE.md's own advice — *"Bring Steels against Tyrannitar sand — they take no sandstorm chip"* — was describing a mechanic the simulator did not have.

Landed as WIRE 31, first in the residual order, which is the real one. Snow is **not** a chip in this generation (Snowscape replaced Hail) and the probe asserts that a fourth arm reads zero, so an engine that chipped in snow would be a new wrong number rather than a wired mechanic. Reads: `sand, Milotic: ability none -10 (a sixteenth is 10), Sand Veil -0; sand, Archaludon (Steel) -0; snow, Milotic -0 . 40.9% of sandstorm boards moved` in the parity above.

The tag over-matched and printing the membership caught it, for the fourth time in this file.

`onImmunity` is Showdown's one hook for "I ignore a named source of harm" and the derivation excluded only the type names:

```
OLD matched 8 : icebody magmaarmor oblivious overcoat sandforce sandrush sandveil snowcloak
NEW matched 6 : icebody overcoat sandforce sandrush sandveil snowcloak
```

Magma Armor's handler is `if (type === 'frz')`; Oblivious's is `if (type === 'attract')`. Wiring it as it stood would have handed a sandstorm immunity to bodies that take the chip. The **weathers are now read out of the handler** — Overcoat refuses both, Sand Veil only sand, Ice Body only snow — so the consumer names no ability. Magma Armor is left with **no tag**, which is honest: its real mechanic is freeze immunity through `onImmunity` rather than `onSetStatus`, so the `statusImmune` derivation next door does not describe it either.

Magic Guard is deliberately NOT exempted and is COUNTED (`fails.magicGuardChip`). It blocks indirect damage through `onDamage`, carries `untagged` (79 uses), and exempting it by name would type a membership list *and* leave the ability half-right, since burn and poison above still chip it.

`priorityMod` **was hollow and the mechanic underneath it is genuinely LIVE**

Prankster's `+1` really is applied in `battleTurn`'s sort. Re-staged behaviourally on the case the wire's own comment names: a **Grimmsnarl** (base 60) puts up Reflect against a **Weavile** (base 125) that is clicking Icicle Crash, so a 0-priority screen goes up after the hit it is meant to blunt. `ability none 118`, Prankster 78 — `x0.667`, which is the doubles screen, so the screen landed first.

`S.lastActs` **is NOT the resolution order and the probe says so in a comment.** `medicham2` writes it from `acts` *before* the sort, so it records what was committed. The first version of this probe printed it and got the same name in both arms, which reads exactly like a dead knob.

A SYSTEMATIC DETECTOR — built, because it was cheap and exact

`tests/test-mechanics.js` now captures each probe's own source and flags any probe that **reads a file instead of running the engine**. It is structural, costs nothing, is written to the census as `hollow`, and the file **exits 1** if it is non-zero — a different exit from MISSING on purpose, because a MISSING mechanic is honest state and a hollow probe is not evidence about the engine at all. All five that ever existed would have been caught the day they were written. **It is 0 now.**

The "both arms agree" version is measured rather than asserted, and the measurement is the reason. `detail` is free-form prose carrying arm values, thresholds ("*a quarter is 43*"), stage counts and stat names all as bare digits, so no parser can tell an ARM from an ANNOTATION. The scan prints how many LIVE probes have ≥ 2 numbers that are all equal:

```
23 over the whole census -> 6 once restricted to LIVE -> 3 after fixing what it found
```

The 6 were not false positives about agreeing arms — they were **one-armed probes**, and three of them could not have failed:

- `preventsStatDrop` read `atk 0 -> 0`, which is also what an engine with no Intimidate prints;
- `blocksStatusMoves` read `target atk stage after Charm: 0 (0 = refused)`, same shape;
- `chargeTurn` read `foe took 0 on the charge turn`, which a move dropped to `kind: pass` also prints — it now also plays turn 2 and demands Fly actually **lands**.

The remaining 3 (`lowersUser -1/-1`, `boostsTarget 2/2`, `statusCategory 0 + par`) are genuine heuristic noise: each asserts a specific non-default value a no-op engine could not produce.

Doing it properly is a **PROTOCOL change** — probes return `arms: {control, test}` and the file asserts `control !== test` — and it has to be applied by hand to all 154 probes, because a probe that kept returning only `detail` would opt itself out silently, which is the same hole in a new place. Costed here so the next pass decides with the number in front of it.

Walking the unprobed tags — 2026-08-04, in descending corpus usage

Tag	Uses	Result
<code>moveClass</code>	76,625	LIVE. Four arms — Iron Fist must boost Mach Punch and must not move Flare Blitz, or "boosts everything" passes
<code>statChange</code>	64,869	LIVE, and the SIZE is asserted: Charm is exactly -2, from the param
<code>sound</code>	14,797	LIVE. Soundproof refuses Hyper Voice and still takes Moonblast
<code>punishesAttacker</code>	8,953	LIVE. Rough Skin tolls Waterfall and not Surf — the trigger is <code>contact</code> and a probe that only tested contact would pass on a wire that punished everything
<code>reflectsStatusMoves</code>	568	WAS MISSING — WIRED (WIRE 33). See below
<code>setsTerrain</code>	141	WAS MISSING — WIRED (WIRE 32). <code>playerAction</code> had a branch for the four weather moves and none for the four terrain moves, so Psychic Terrain (114 uses) resolved to <code>kind: pass</code> . Probed by outcome — the foe's Ice Shard is blocked — not by reading the field back, which would pass on a string nothing reads
<code>terrainScaled</code>	296	WAS MISSING — WIRED (WIRE 34). Expanding Force 105 → 157

`reflectsStatusMoves` **over-matched, and this is the fifth membership print to earn its keep.** `onAllyTryHitSide` is the hook for "I react to something aimed at my side" and says nothing about what the reaction is:

```
OLD matched 3 : magicbounce, sapsipper, soundproof
NEW matched 1 : magicbounce
```

Sap Sipper **boosts** off an ally's Grass move; Soundproof **refuses** an ally's sound move. Wiring the tag as it stood would have sent every Will-O-Wisp aimed at a **Soundproof** body back at its user — 355 corpus uses — and a bounce is strictly worse than an immunity, because it is a move that lands on you. The discriminator is the bounce itself: only Magic Bounce rebuilds the move and calls `useMove` back at the source, gated on Showdown's `reflectable` flag.

`reflectable` **was added to the** `moveClass` **derivation in the same pass**, so the wire is the same ability-names-a-flag / move-carries-it JOIN `immuneToMoveClass` already uses rather than a second membership rule in the consumer. 60 moves carry it; the tags.json diff was verified to be **exactly** that addition and nothing else. Probe reads `atk stages (target/user): ability none -2/0, Magic Bounce 0/-2`. What is **not** modelled is stated: the tag's `scope` covers the whole side including hazards, and this engine keeps neither hazards nor side conditions.

`terrainScaled` **carried no number and that is why nothing read it.** `{scalesWith:'terrain'}` named the mechanism and gave a consumer nothing, so Expanding Force (182) and Rising Voltage (114) were priced at base power in every rollout. `tag_dex` now pulls **which terrain** out of Showdown's own `isTerrain("psychicterrain")` and **the multiplier** out of the `chainModify` or `basePower * n` beside it — and it probes `onBasePower` too, which was never probed and is where Expanding Force and Misty Explosion live. Membership: `expandingforce 1.5, risingvoltage 2, mistyexplosion 1.5`, and `terrainpulse` **keeps the**

bare tag with no number on purpose — it changes TYPE as well as power and must not be given a multiplier. Grounded-ness is not tracked (the same caveat `priorityRefusedAbove` already carries) and Expanding Force becoming a spread move is not modelled; both are stated in the code.

Every regeneration of `data/tags.json` was verified the way this file requires — diff excluding `uses`, count the entries, read every one. `weatherChipImmune` 8, `reflectsStatusMoves` 3, `moveClass` 60, `terrainScaled` 3, all intended. `engine/feature_fixture.js --check` exits **0** after each, so **no refit is owed**.

Filed, not fixed

- **A THIRD terrain vocabulary sits between the Board and the leaf, and it finds nothing.** `engine/miltank.js:781` and `engine/rollout_r1.js:175` both extract with `['electric','grassy','misty','psychic'].find(t => board.hasField(t))`, while `board.startField` stores `norm(move.terrain)` — the LONG words. Measured: **0 of the 863 terrain-carrying boards** in a 69,623-board walk are found by that extractor, so the leaf is handed `''` on every board that has a terrain. `medicham2` now accepts either vocabulary, so the fix on their side is to pass the key the Board actually holds. `miltank.js` and `rollout_leaf.js` are **SEARCH's**; `rollout_r1.js` is a MEASURE gate. Exposure is small (1.24% of boards) and the fix is one array.
- `engine/position_features.js:296` **reads the LONG terrain words and :291 the untranslated weather.** Both change a **feature vector**, which is the refit edge MEASURE owns. `board.js:1190` holds a third copy of the weather map (`WEATHER_KIND`, sun/rain only). Neither file is ENGINE's.
- `engine/status.js` **prints the differential count without its seed.** The artifact now carries `seed`, `requested`, `skipped_multihit` and `skipped_non_finite`; the print reads none of them, so "1/400 differential comparisons disagree" still looks unconditional and does not say that 15 rows were skipped as not-comparable. `status.js` is MEASURE's file. One line.
- `engine/status.js` **does not read `data/interaction-matrix.json` at all**, so the ENGINE block prints a census and a damage residual and says nothing about whether the mechanics work TOGETHER — which is now a bigger surface than either (1,008 live pairs against 174 probes). The artifact carries `live`, `agree`, `inert`, `saturated`, `ko_timing`, `threw` and the full `dropped_by_the_generator` ledger, ready to print. `status.js` **is MEASURE's file**. Filed, not fixed.
- **The last differential row is a LAYER MISMATCH, not an engine bug, and it is flagged in place.** `chesnaught woodhammer -> mimikyu` reads `showdown 0-0`, `medicham 120-130` and is marked SUSPECT. Showdown's `onDamage` returns 0 while the maxhp/8 never lands, because this harness never calls `battle.update()`; MEDICHAM's `dmgRange` correctly reports raw damage because WIRE 23 substitutes one level up in the battle loop. Both engines are right and the comparison is asking `dmgRange` a question about `battleTurn`. It is still COUNTED in the residual — flagging must never move the number. Fixing it properly means teaching the harness to run the damage-layer abilities, which is a bigger change than the row is worth.
- `battleResult` **cannot tell a finished battle from an expired clock.** `medicham2-browser.js:1802` scores bodies-then-HP unconditionally; `battleOver` returns true for a wipeout *and* for `S.turn >= maxTurns`, and the caller cannot distinguish them from the return value. Every cap-expired payout is therefore a material count returned as a win probability, silently. Filed by SEARCH 2026-08-04 while testing whether that was the cause of the flat leaf calibration. **It is not** — measured by wrapping `battleResult` over 1.1M payouts, 99.5–99.8% end by an actual wipeout at every explore setting and at horizons 20 and 60, and cap-hits run 0.2–0.5%. So this is a latent hazard, not a live defect, and it is filed rather than ranked. The cheap fix is for `battleResult` to return the reason beside the score so a caller can weight or discard those rows; do not change what it *scores*, which several artifacts depend on.

The authorised list — ALL LANDED, 2026-08-04

Cleared after the SEARCH explore sweep and the leaf calibration landed. Census **42** → **100 live**, differential **4/400** → **1/400** at seed 20260804, refit edge still CLEAN (all 58 feature columns hash-identical, so no refit is owed).

The item ranked first was not a bug. `redirects` (7,240 uses) was filed as "the attack VANISHES — the worst bug in the repo". It does not. The probe aimed **Dragon** Claw at **Whimsicott**, which is Grass/**Fairy** and immune to Dragon: Follow Me fired correctly, pulled the attack off Incineroar, and landed it on a body that takes exactly zero. Both arms read 0 and the conclusion was written from that. Re-staged with Milotic the same code reads `aimed 0 / redirector 101`. Follow Me and Rage Powder have always worked, so **no rollout, H2H, R3 or R4 result is invalidated by this** — the blast-radius note attached to the original filing should be retracted.

Nine probes in this file have now been wrong before the engine was, and this is the first that was believed. A red probe is a QUESTION.

#	Item	Result	
1	<code>~~ redirects ~~ → redirectsType</code> (Lightning Rod, 1,901)	LANDED (WIRE 25). The real redirection gap: the engine only looked for the Follow Me volatile, so an Electric move aimed past a Lightning Rod hit its partner. Probe now reads <code>aimed 0 / rod 0 / spa +1</code> — the rod both draws and absorbs, and the boost is the receipt.	
2	<code>drain</code> (8,553)	LANDED (WIRE 19). <code>dealt 51 → user 85→110</code> . The fraction did not exist in the artifact: the tag said <code>readFrom:"m.drain"</code> , a pointer into a dex this engine does not have. <code>tag_dex.js</code> now emits the value, so Draining Kiss gets its 3/4 instead of an assumed 1/2.	
3	<code>multiHit</code> (4,655)	LANDED (WIRE 20). <code>expectedHitsOf()</code> already existed and only <code>punishExposure</code> read it. Rock Blast 17 → 52. The differential now SKIPS multi-hit moves — comparing an expectation against one sample is not a comparison — so <code>tests/test-mechanics.js</code> is the only guard and says so in its own comment.	
4	<code>choiceLock</code> (5,886)	LANDED (WIRE 24), in <code>medicham2</code> — no <code>board.js</code> change was needed . <code>chooseAction</code> had honoured the lock since WIRE 18; <code>`_a = forced \</code>	<code>\ chooseAction(...)` let every caller-supplied action through. A switch is still legal, which is the half a naive fix breaks.</code>
5	<code>fixedDamage</code> (1,122)	LANDED (WIRE 21). Super Fang, Final Gambit, Endeavor and the OHKOs had no base power, so <code>hasPower()</code> rejected them and they were worth zero. Counter/Mirror Coat/Metal Burst need turn state a pure pricing function is not given, and are left at zero loudly rather than approximated.	
6	Foul Play (734)	LANDED . <code>dmgRange</code> read <code>statSwap</code> (Body Press, Psyshock) and nothing had ever read <code>swapsStat.offensiveFrom</code> . The target's Attack stage moves with it, or a Swords Dance matchup — where the move is actually played — becomes a new wrong number. Differential: both directions now rel 0.0% .	
7	<code>immuneToMoveClass</code> (Soundproof 349, Overcoat 240, Bulletproof 85)	LANDED (WIRE 22) with CONTROL FIX 10 in the same pass, as required. Membership printed first: five abilities, and <code>magicbounce / reflectable</code> is excluded deliberately — it bounces status moves and grants no damage immunity. Powder is left to <code>powderBlocked()</code> , which already owns that question.	
8	Disguise	LANDED (WIRE 23). Exactly <code>maxhp/8</code> , not a flat zero. My probe's own threshold (<code>≤15% of the real hit</code>) would have rejected the correct fix — 16 against a 92 hit is 17% — so the assertion was corrected	

#	Item	Result
		to the exact rule first. The busted flag is deliberately NOT cleared on switch-out.
9	Dry Skin Fire x1.25	LANDED — tag first, exactly as specified. <code>tag_dex.js</code> now probes <code>onSourceBasePower</code> beside the stat route; membership printed before wiring matched exactly one ability corpus-wide. The four hardcoded <code>thickfat/heatproof/purifyingsalt/waterbubble</code> lines in <code>dmgRange</code> are gone, replaced by one tag-driven read. Differential: <code>houndoom fireblast -> heliolisk</code> now rel 0.0% .
—	<code>train_policy.js</code> <code>writeWeights</code> <code>provenance</code>	STILL NOT DONE. Carried from the previous pass.

Regenerating `data/tags.json` **was verified, not assumed.** The generator did **not** reproduce its own artifact on the first run — 285 entries differed. Every one of those was the `uses` count alone, because the store grew mid-session; after excluding `uses`, exactly **9** entries changed and all 9 were the intended ones (8 drain moves + `dryskin`). No feature reads `.uses`, and `feature_fixture --check` is clean afterwards. Anyone regenerating tags should run that same diff rather than trusting the file.

The mechanism in item 5 was diagnosed wrongly first and the correction matters. A starved `basePowerCallback` does compute NaN, but Showdown clamps before it reaches the target's HP, so the row comes back as a clean, plausible, entirely fake **zero** — not a NaN. A `Number.isFinite` guard therefore does *not* catch it and has never fired on this corpus; the guard is kept and says so in its own comment, and the phantom zero is caught by the SUSPECT marker instead. A fix aimed at the wrong mechanism is still a bug.

Ranked engine fixes — every one has a red probe behind it

The 2026-08-04 tag walk added 88 probes and moved the census from 54 probed to 142. `live` went 42 → 100 and `missing` from 12 to 42, and not one previously-live probe fell. Nine of the entries below have since LANDED (see the authorised list above); what remains here is the queue. Ranked by corpus uses, then by how badly wrong the behaviour is, with Lesson 3 applied by hand.

Nine of these were my probe being wrong, not the engine, and each was caught by its own control before it reached this list — a spread move aimed where a single-target one was needed, Close Combat fired at a Ghost that is immune to it, Toxic fired at a Steel that cannot be poisoned, and a Fly declared by a Pokemon slower than its attacker (which the real game also lets through). That is Lesson 5 for the sixteenth-through-nineteenth time, and it is why every probe here prints BOTH arms.

- redirects — 7,240 (Rage Powder 5,874, Follow Me 1,366). Take this first, and not for the usage.** The failure is worse than absence: the attack *vanishes*. Probe reads `no Follow Me: aimed 92 / partner 0 | Follow Me: aimed 0 / partner 0` — nobody takes it. Every Rage Powder in every rollout ever run has been a free team-wide Protect, and the searcher maximises exactly the lines its model is most optimistic about (Lesson 2). `redirectsType` (Lightning Rod, 1,901) is the same bug through a different door and reads the same way.
- drain — 8,553 (Matcha Gotcha 4,957, Giga Drain 1,255, Drain Punch 916, Draining Kiss 814).** move dealt 51 to the foe; user 85 -> 85 hp. The damage lands and the heal is dropped, so the most-clicked recovery route in the format is worth nothing.
- choiceLock — 5,886.** Not unimplemented: `tests/test-choice-lock.js` asserts it four ways and passes, on `board.js`. MEDICHAM obeys whatever action it is handed. Two engines disagreeing about a FACT is the CLAUDE.md rule this project has broken most expensively.
- multiHit — 4,655 (Dual Wingbeat 2,675, Twin Beam 676, Triple Axel 522, Population Bomb 385).** Priced as ONE hit. Dual Wingbeat is exactly half its real damage; Population Bomb about a seventh.

The differential cannot see this — single-call `moveHit` also hits once — so only the probe can.

5. `blocksSoundMoves` — **2,726 (Throat Chop)**. One move, all real clicks, no effect at all.
6. `punishesContact` — **1,761 (Spiky Shield 887, Baneful Bunker 698, King's Shield 176)**. The block works and the punish does not, so the three are currently just Protect.
7. `swapsStat` / **Foul Play — 734, and the only red item with an independent authority behind it**. Confirmed twice against Showdown in the same run and in BOTH directions — `spiritomb foulplay -> wyrdeer` 178 there against 132 here, `klefki foulplay -> pangoro` 19 against 8. `dmgRange` reads the `statSwap` tag, which Foul Play does not carry; its `swapsStat` params say `offensiveFrom:"target"` and nothing reads that field.
8. `overridesEffectiveness` / **Freeze-Dry — 1,252**. `mrrime freezedry -> araquanid` reads 96-114 on Showdown and 24-28 here: the full 4x, the move's entire identity, and independently confirmed.
9. `fixedDamage` — **1,122 (Super Fang 577, Final Gambit 250, Endeavor 93)**. `mv.bp=0`, so `dmgRange` short-circuits and these moves are worth literally zero to the engine.
10. `boostsEachTurn` — **1,137 (Speed Boost 700, Moody 434)** and `healsOnSwitchOut` — **1,057 (Regenerator 830)**. Both sheet counts, but both fire on a schedule rather than on a condition, so the count is close to the real rate.
11. `costsUserHP` — **929 (Substitute 528)**. Resolves to `kind: affect` and costs nothing; Substitute is not modelled at all and the HP cost is only its visible half.
12. `poisonsOnMyContact` — **947 (Poison Touch)**, `reducesAllyDamage` — **875 (Friend Guard)**, `immuneToMoveClass` — **838 (Soundproof 344, Overcoat 240, Bulletproof 84)**. Sheet counts, and Lesson 3 bites hardest on Poison Touch: it fires only on CONTACT moves, so a special attacker carrying it contributes to the 947 and never triggers.
13. `clearsBoosts` — **542 (Haze)**. `playerAction` resolves it to `kind: pass`, so Haze is a wasted turn in every rollout.
14. `partialTrap` **772**, `terrainScaled` **296**, `cantUseTwice` **186**, `untagged` / **Marvel Scale**. Real but small. Infestation lands its 8 damage and then chips nothing at all; Spiky Shield (item 6) blocks correctly and then punishes nothing — both probes print which half happened, so neither can be confused with a move that never resolved.
15. `formeChange` / **Disguise — 111 sheet uses, and the count badly understates it**. Mimikyu appeared in three of the residual differential rows across four seeds, because every Mimikyu that appears at all uses Disguise on turn one. Note the artifact will not hand this to you: `disguise.tags` is `["preventsCrit","formeChange"]`, and `preventsCrit` also holds Battle Armor, Shell Armor and Ice Face.

The second block, found by walking further down the same list. Smaller individually, and three of them are whole categories of turn the engine cannot represent at all:

1. `forcesSwitch` — **513 (Roar 400, Dragon Tail 96)**. The move deals its 69 damage and the drag does not happen, so a phazing turn is priced as a weak attack.
2. `noRecoil` — **731 (Rock Head)**, `curesVolatile` — **665 (Mental Herb)**, `blocksExplosion` — **545 (Damp)**, `ignoresTypeImmunity` — **469 (Scrappy)**, `reordersTurn` — **463 (After You 148, Instruct 162, Quash 153)**. All sheet or click counts, all read identical across the knob.
3. `userFaints` — **338**. Explosion leaves its user on full HP. `crashOnMiss` — **199**: High Jump Kick misses correctly and costs nothing. `hazard` — **195**: the switch happens and Stealth Rock chips a 4x-weak Staraptor for zero.
4. `typeBecomesMoveType` — **248 (Protean)**, `ignoresDefenderAbility` — **212 (Mold Breaker, verified against a Levitate hard zero)**, `blocksHealing` — **196 (Psychic Noise)**, `curesStatus` — **208 (Lum Berry 175)**.

5. `multiAccuracy` — **907 (Triple Axel 522, Population Bomb 385), and it hides a second bug.** Three 90% rolls compound to about 73%, and `moveAccuracy('tripleaxe1')` returns **100** — so the printed accuracy is wrong before the per-hit rule is even considered.
6. `alwaysCrit` — **274 (Flower Trick 216).** Same design question as `critRatioUp` below, not a separate decision.

THE CRIT VERDICT — it WAS a bug, it is fixed, and here is the exposure. 2026-08-04.

The previous pass wrote "`critRatioUp` *may not be a bug* — `dmgRange` *models no crit anywhere*". That was half right in a way that hid the real defect: **the BATTLE LOOP has always rolled a crit**, a flat `rng() < 1/24` for every move and every defender. So the engine had two crit facts and both were wrong, in opposite directions.

Measured before anything was touched, the way the terrain vocabulary was. Over **48,274 stored games**, **7.53%** carry a crit-tag move on an observed set — **1.68%** an `alwaysCrit` move and **5.98%** a `critRatioUp` one. `preventsCrit` outside Disguise (Shell Armor, Battle Armor, Ice Face) is **41 games, 0.08%**. By clicks: `alwaysCrit` **278** (Flower Trick 219, Frost Breath 40, Storm Throw 19), `critRatioUp` **1,162** (Psycho Cut 276, Leaf Blade 169, Stone Edge 169, ...).

The two halves are not the same size of error and do not belong in the same place.

- `alwaysCrit` is a **certainty**: a flat x1.5 the pricer was missing on every one of 278 clicks, so Flower Trick was priced 33% below what it does. It belongs in `dmgRange`, and it is what Showdown's own `willCrit` does — so the differential AGREES with it once its control is right.
- `critRatioUp` is a **RATE**, 1/24 to 1/8, an expectation difference of about 4%. It must NOT go in `dmgRange`: folding an expectation into a min/max stops `max` being the maximum roll and puts every ratio move permanently out of step with the differential's non-crit comparison. It rides the battle loop's roll, which is where the 1/24 already lived.
- **Shell Armor did nothing whatever**, and now turns both off.

Landed as **WIRE 35**, with one `critChance()` that every caller reads. The two ABILITY carriers of `critRatioUp` are **deliberately refused and counted** (`MEDFAILS.critRatioAbility`, 15 corpus uses): the tag's only param is `critRatio: 2`, which cannot express Merciless's condition — a GUARANTEED crit into a poisoned target, not a permanent stage bump — and Super Luck, which genuinely is a permanent bump, is indistinguishable from it in the artifact. Wiring both would hand Merciless an unconditional 1/8 it never has. Scope Lens (an ITEM, unconditional) IS wired.

THE PROBE HAD TO BE REWRITTEN, and that is the actual answer to "is it a bug". It read `dmgRange(Night Slash) > dmRange(the same move with its id changed)` — it was asking the PRICER for an expectation, which is the thing that must not happen. It is now behavioural and pinned at a roll that SEPARATES the rates: **0.1 is below 1/8 (0.125) and above 1/24 (0.0417)**, so a base-rate move cannot crit on it and a one-stage move must. Four arms, because two cannot attribute it — Shell Armor is the discriminator on the ratio move, and **Crunch** (Dark, physical, same attacker, same target, no crit ratio) is the control that must not move at all. An engine that simply raised the base rate for everything passes a two-armed version and fails this one. Reads `Night Slash plain 100 / Shell Armor 67 | Crunch plain 77 / Shell Armor 77`.

THE DIFFERENTIAL WENT 1/400 TO 5/400 THE MOMENT THE ENGINE LEARNED THE MECHANIC, AND THE HARNESS WAS WRONG. Four of the five new rows were Flower Trick and Frost Breath with MEDICHAM exactly 1.5x above the reference, because `test-engine-diff.js` pinned `move.willCrit = false` — right for a random crit, which is noise both engines must be held to, and wrong for

the three moves whose crit is not random at all. **CONTROL FIX 11** pins it to the move's own dex value, so it now only ever CLEARS a crit that would otherwise be rolled. Back to **1/400** at seed 20260804, same single SUSPECT row.

preventsCrit **(151 uses) had never been probed at all** and now is, through the sharpest available form: Flower Trick's max into a plain Garchomp against the same into a Shell Armor one, with a third arm carrying no crit tag to show the plain number really is the un-crit one. plain 123, Shell Armor 82, no crit tag 82.

One that is still NOT a one-line fix, called out so nobody starts it by accident:

- `writesAccuracy` **(987)** and `accuracyMod` **(927)** are blocked on a signature, and the cost is now stated rather than gestured at. `moveAccuracy(id, field)` takes neither the attacker nor the defender, so No Guard, Compound Eyes, Sand Veil and Snow Cloak have nowhere to be read from. Both probes clear their control — the No Guard one pins the roll at 0.9 against 80% accuracy, so the control correctly MISSES — and both still read identical across the knob, which by Lesson 5 means unwired rather than unimportant. **What the change costs: 11 call sites across 4 files.** Inside `medicham2-browser.js`, `moveAccuracy` is read by `playerAction` (which has both bodies), the battle loop's to-hit roll (both), the `affect` branch's status roll (both), the status branch (both), `bestMoveVs` (both) and `expectedHitsOf` (**neither** — it is called from `dmgRange`, which is pure and is handed no field). `engine/exposure.js`, `engine/board.js` and `engine/position_features.js` each call it too, and the last two are **not ENGINE's** — a signature change there is a feature-vector change and therefore the refit edge MEASURE owns. The compatible shape is `moveAccuracy(id, field, att, def)` with both optional, which leaves every existing caller correct and lets the six that have the bodies pass them; `expectedHitsOf` would keep the two-argument form and the ability would simply not apply there, which is honest because a pure pricing function has no attacker. **It is a deliberate pass, not a one-liner, and it should not be started inside another division's run.**
- `needsTargetToAttack` / **Avalanche** — **VERDICT: the probe asks for a rule that does not exist, and it is left MISSING on purpose.** Avalanche doubles when the USER was damaged BY THAT TARGET THIS TURN. The probe compares a fresh body against one whose `curHP` was halved — i.e. it asks `dmgRange` to double on "the user is below full HP", which is not the mechanic and would be a new wrong number on every hurt Avalanche user. `dmgRange` is handed no turn state and must not invent any; the tag's own param is the prose string "target attacking". **13 corpus uses.** The nine other members of the tag include Sucker Punch (6,673), which is already fully modelled through `failsIfTargetNotAttacking` — so the tag is not inert, only this member is.

Ordering the queue

`tests/mechanics_rank.js` ranks unread tags by corpus usage — use it, not intuition, and read the result against Lesson 3: **usage counts are sheet counts.** Blaze reads 4,585 uses and is worthless because 30 of 54 entries are a Charizard that megas into Drought on turn one. Ice Scales, Filter, Aerilate, Prism Armor, Punk Rock and Ripen read zero.

The damage disagreements in the generated block are ordered by `uses` for the same reason, and carry the same caveat.

Before wiring a new derived tag

Print what it matched. Every derivation over-matches on the first try — `refusesStatusMoves` caught Telepathy and Wonder Guard, `speedOnItemLoss` caught Sticky Hold, `failsIfTargetNotAttacking` caught Quick Guard, Wide Guard and Round. See `docs/LESSONS.md` §4.

Done looks like

- The census `live` count is higher than it was.
- No probe that passed yesterday fails today.
- The differential test finds fewer disagreements **at the same** `--seed` **and** `--n`, or the same ones with smaller error. Quoting a residual without its seed is quoting a coin flip: the sampler used bare `Math.random()` until 2026-08-04, and two runs on identical source gave 6 and then 3.
- Nothing in the hand list above that has not become a probe.

`missing` **going UP is not a regression, and this is the one place the rule is easy to misread.** It rose $12 \rightarrow 42$ on 2026-08-04 while `live` rose $42 \rightarrow 100$, because 88 probes were written for mechanics nobody had asked about before. The number that may never fall is `live`. A rising `missing` means the census stopped flattering the engine.

The one thing this division owes the others

A **named engine release**. Fixes batch; the release is what triggers the refit and the restamps — see [DIVISIONS.md](#). Landing engine changes continuously is what leaves SEARCH measuring a build that no longer exists.